CO226: Database Systems

Structured Query Language (SQL)

Dr. Sampath Deegalla
Senior Lecturer, Department of Computer Engineering
University of Peradeniya
sampath@eng.pdn.ac.lk

3rd June 2025

Structured Query Language (SQL)

SQL (Structured Query Language)

- SQL is a comprehensive database language.
- It supports both
  - DDL (Data Definition Language)
  - DML (Data Manipulation Language)
- It includes features such as
  - views
  - security
  - authorization
  - transaction control
  - embedding SQL

SQL vs. Relational algebra/calculus

- SQL includes some features from relational algebra and it is greatly based on tuple relational calculus
- SQL is user friendly than both relational algebra and calculus
- In relational algebra, user should specify how to execute the query operations
- In SQL, user specifies what the result is to be

Terminology

SQL	Relational Model
table	relation
row	tuple
column	attribute

CREATE SCHEMA

- Specifies a new database schema by giving it a name
  - `CREATE SCHEMA COMPANY;`
  - In MySQL the following statments can be used to create a database schema
    - `CREATE SCHEMA COMPANY;`
    - `CREATE DATABASE COMPANY;`
    - CREATE SCHEMA is a synonym for CREATE DATABASE

CREATE TABLE

- Specifies a base relation (table) by giving it a name, specifying each of its attributes and initial constraints
- Each attribute is given a name and a data type to specify its domain of values
- A constraint `NOT NULL` may be specified on an attribute to disallow null values

```
CREATE TABLE DEPARTMENT (
DNAME VARCHAR(10) NOT NULL,
DNUMBER INTEGER NOT NULL,
MGRSSN CHAR(9),
MGRSTARTDATE DATE );
```


CREATE TABLE

- Key attributes can be specified via the `PRIMARY KEY`
- Use `UNIQUE` phrases to indicate alternate keys/unique values
- Use `FOREIGN KEY` to specify referential integrity

```
CREATE TABLE DEPARTMENT (
DNAME VARCHAR(15) NOT NULL,
DNUMBER INTEGER NOT NULL,
MGRSSN CHAR(9) NOT NULL,
MGRSTARTDATE DATE,
PRIMARY KEY (DNUMBER),
UNIQUE (DNAME),
FOREIGN KEY (MGRSSN) REFERENCES EMPLOYEE(SSN)
);
```

Attribute Data Types in SQL

- The basic data types include numeric, character string, bit string, boolean, date and time.
- Numeric data types

  INTEGER/INT and SMALLINT integer numbers of various sizes

  FLOAT/REAL and DOUBLE PRECISION floating-point (real) numbers of various precision

  DECIMAL(i,j)/DEC(i,j) or NUMERIC(i,j) formatted numbers: i - decimal digit, j-number of digit after decimal point

Attribute Data Types in SQL

- Character-string data types

  CHAR(n)/CHARACTER(n) fixed length character-string: n-number of characters

  VARCHAR(n)/CHAR VARING(n)/CHARACTER VARYING(n) varying length character-string: n-maximum number of characters

  CHARACTER LARGE OBJECT/CLOB large text values, e.g., documents. In mysql TEXT data type is used

Attribute Data Types in SQL

- Bit-string data types

  BIT(n) fixed length bit string

  BIT VARYING(n) varying length bit string (Not supported in MySQL)

  BINARY LARGE OBJECT/BLOB large binary values, e.g., images (MySQL variants: TINYBLOB, BLOB, MEDIUMBLOB, LONGBLOB)

Additional Data Types

- BOOLEAN for TRUE/FALSE values
- DATE

  DATE Made up of year-month-day in the format yyyy-mm-dd

  TIME Made up of hour:minute:second in the format hh:mm:ss

  TIME(i) Made up of hour:minute:second plus i additional digits specifying fractions of a second. format is hh:mm:ss:ii...i

  TIMESTAMP Has both DATE and TIME components

Boolean and Temporal Data Types in SQL (MySQL)

- **BOOLEAN**: Used for `TRUE/FALSE` values
  - In MySQL: `BOOLEAN` is an alias for `TINYINT(1)`
  - `0 = FALSE, 1 = TRUE`
- **DATE**: `yyyy-mm-dd`
- **TIME**: `hh:mm:ss`
- **TIME(*i*)**: `hh:mm:ss.iii...`
  - Fractional seconds precision (i) ranges from 0 to 6
- **TIMESTAMP**: Combines `DATE` and `TIME`
  `YYYY-MM-DD hh:mm:ss[.fraction]`
  - Timezone-aware in MySQL
  - Often used for auto-tracking row creation or update time

Additional Data Types

- INTERVAL

  INTERVAL Specifies a relative value rather than an absolute value. Can be DAY/TIME intervals or YEAR/MONTH intervals. Can be positive or negative when added to or subtracted from an absolute value, the result is an absolute value.
- Note: not available in MySQL

Specifying constraints in SQL

- `NOT NULL` to indicate null is not permitted for a perticular attribute
- `DEFAULT` to define a default value for an attribute e.g. `DEFAULT <value>`
- `PRIMARY KEY` to specify one or more attributes that make up the primary key of a relation
- `UNIQUE` to specify alternate (secondary) keys (i.e. candidate keys in the relational model) e.g. `Dname VARCHAR(15) UNIQUE;`
- `FOREIGN KEY` to specify referential integrity constraints
- `CONSTRAINT` to give name to constraints

Referential Integrity Options

- We can specify RESTRICT, CASCADE, SET NULL or SET DEFAULT on referential integrity constraints (foreign keys)
- This must be specified with either ON DELETE or ON UPDATE
- RESTRICT reject update operation that will cause a violation
- SET NULL set to null value
- CASCADE propaget the update
- SET DEFAULT set to default value (Not supported in MySQL)

REFERENTIAL INTEGRITY OPTIONS

- Examples

```
CREATE TABLE DEPT (
DNAME VARCHAR(15) NOT NULL,
DNUMBER INTEGER NOT NULL,
MGRSSN CHAR(9),
MGRSTARTDATE DATE,
PRIMARY KEY (DNUMBER),
UNIQUE (DNAME),
FOREIGN KEY (MGRSSN) REFERENCES EMP(SSN)
ON DELETE SET DEFAULT ON UPDATE CASCADE
);
```


REFERENTIAL INTEGRITY OPTIONS

```
CREATE TABLE EMP (
ENAME VARCHAR(30) NOT NULL,
ESSN CHAR(9),
BDATE DATE,
DNO INTEGER DEFAULT 1,
SUPERSSN CHAR(9),
PRIMARY KEY (ESSN),
FOREIGN KEY (DNO) REFERENCES DEPT
ON DELETE SET DEFAULT ON UPDATE CASCADE,
FOREIGN KEY (SUPERSSN) REFERENCES EMP(SSN)
ON DELETE SET NULL ON UPDATE CASCADE
);
```


DROP TABLE

- Used to remove a relation (base table) and its definition
- The relation can no longer be used in queries, updates, or any other commands since its description no longer exists
- Example: `DROP TABLE DEPENDENT;`

ALTER TABLE

- Used to add an attribute to one of the base relations
- The new attribute will have NULLs in all the tuples of the relation right after the command is executed; hence, the NOT NULL constraint is not allowed for such an attribute
- Example: `ALTER TABLE EMPLOYEE ADD JOB VARCHAR(12);`
- The database users must still enter a value for the new attribute JOB for each EMPLOYEE tuple. This can be done using the UPDATE command.